

# GitHub Innovation Portal

## A Real-Time Open-Source Trend Analytics Platform

Jann Erhardt

erharjan@students.zhaw.ch

Zurich University of Applied Sciences  
Winterthur, Zurich, Switzerland

Gian Gamper

gameggia@students.zhaw.ch

Zurich University of Applied Sciences  
Winterthur, Zurich, Switzerland

Jonas Bratschi

bratsjon@students.zhaw.ch

Zurich University of Applied Sciences  
Winterthur, Zurich, Switzerland

### ABSTRACT

Open-source activity on GitHub generates tens of millions of events per day, making it difficult for developers, researchers, and technology scouts to identify promising early-stage projects before they gain mainstream attention. This paper presents the *GitHub Innovation Portal*, an end-to-end streaming analytics platform that ingests the public GitHub event stream in near-real time, enriches each event with geographic and user-profile metadata, stores it in a time-series optimised database, and exposes actionable insights through two complementary visualisation layers.

The system is built on Apache Kafka (KRaft) for durable event buffering, TimescaleDB for compressed hypertable storage with pre-computed continuous aggregates, FastAPI for a low-latency REST and Server-Sent Events interface, and a Next.js 16 dashboard for live trend visualisation. The entire stack is containerised and deployable with a single `docker compose` command. **[TODO: Refine after report completion]**

### CCS CONCEPTS

• **Information systems** → **Data streams**; *Database management system engines*; • **Software and its engineering** → Software architectures.

### KEYWORDS

stream processing, GitHub event API, Apache Kafka, TimescaleDB, real-time analytics, open-source trend detection, geo-enrichment, Server-Sent Events, Next.js dashboard

#### ACM Reference Format:

Jann Erhardt, Gian Gamper, and Jonas Bratschi. 2025. GitHub Innovation Portal: A Real-Time Open-Source Trend Analytics Platform. In *Proceedings of Big Data Projects (PM4 FS25)*. ACM, New York, NY, USA, 7 pages.

## 1 INTRODUCTION

The open-source ecosystem hosted on GitHub grows continuously, with millions of push events, pull requests, issue comments, and repository forks published every day [2]. For developers, researchers, and technology scouts this volume presents a paradox: the very richness of the signal makes it practically impossible to manually track emerging projects before they appear in mainstream media.

Existing discovery mechanisms rely on lagging indicators — stars accumulated over weeks, blog posts, or curated lists — rather than on the raw momentum visible in the live event stream. A repository that doubles its commit frequency or attracts contributors from three continents in a single day represents a signal of genuine innovation; today that signal is largely wasted.

This project addresses the gap with the *GitHub Innovation Portal*: an end-to-end streaming analytics platform that continuously ingests the GitHub public events API, enriches each event with geographic and user-profile context, persists the data in a time-series optimised store, and surfaces trends through an interactive dashboard and operational Grafana dashboards.

The remainder of this paper is structured as follows. Section 2 surveys related approaches. Section 3 describes the system architecture. Section 4 details the implementation of each component. Section 5 presents evaluation results. Section 6 justifies the technology choices. Section 7 discusses challenges encountered. Section 8 concludes with lessons learned and future work.

## 2 RELATED WORK

**TODO:** Insert 3–5 brief paragraphs covering related systems and prior art. Suggested topics:

- GitHub Archive / GH Archive project (historical event download)
- GHTorrent / Software Heritage (large-scale repository mining)
- Academic papers on streaming architectures (e.g., Lambda/Kappa architecture, Kafka Streams)
- Open-source trend detection tools (e.g., Trending page on GitHub, third-party trackers)
- Time-series databases: comparison papers (InfluxDB vs. TimescaleDB)

Add corresponding `.bib` entries to `references.bib`.

**[TODO: Write Related Work section (approx. 300–400 words).]**

## 3 SYSTEM ARCHITECTURE

**[TODO: Insert architecture diagram here.]**

Export the ASCII diagram below as a vector figure (e.g., draw.io PDF).

**Figure 1: End-to-end architecture of the GitHub Innovation Portal. Events flow from the GitHub API through Kafka into TimescaleDB, where they are served by FastAPI to the Next.js dashboard and Grafana.**

The architecture of the project consists of a real-time data streaming pipeline designed to ingest, enrich, and visualize GitHub events. It is built to efficiently manage GitHub API rate limits and ensure

high scalability by decoupling individual components using a message broker.

### 3.1 Data Ingestion (Producer)

The **Producer** (`producer/producer.py`) serves as the entry point of the pipeline:

- **Polling:** It polls the public GitHub Events API every 10 seconds.
- **Deduplication:** To prevent redundant data, the producer filters events based on their unique GitHub Event ID.
- **Kafka Integration:** Cleaned raw JSON data is published to the Kafka topic `github.events.raw`.
- **Rate Limit Management:** A standard GitHub personal access token has a rate limit of 5 000 requests per hour, which is sufficient for continuous polling without hitting the unauthenticated cap of 60 requests per hour.

### 3.2 Message Broker (Kafka)

**Kafka** acts as the central message bus, operating in KRaft mode without the need for Zookeeper:

- **Decoupling:** It separates the high-frequency data ingestion (Producer) from the more complex data enrichment steps (Consumer).
- **Durability:** Kafka serves as a reliable buffer; if a consumer fails, data is retained for 48 hours, allowing the system to resume where it left off.
- **Scalability:** The main topics are divided into three partitions, enabling parallel processing by multiple consumer instances.

### 3.3 Processing and Enrichment (Consumer & Geocoder)

In this stage, raw events are transformed into high-value data through a highly optimized enrichment process. The system is designed to maximize throughput while respecting strict external API constraints.

- **Parallel Consumer Instances:** The system is designed for horizontal scalability. Multiple consumer instances (e.g., three instances in multi-consumer mode) can run in parallel. Using Kafka's partition assignment, each instance is "pinned" to specific partitions, ensuring that event processing is distributed evenly and without overlap.
- **Token Pool Management:** To overcome GitHub's rate limits, the system implements a **TokenPool** mechanism. It manages multiple API keys for both user and repository metadata. The pool uses a round-robin strategy and automatically skips tokens that have been flagged as rate-limited, resuming their use only after their specific reset time has passed.
- **GraphQL Batch Processing:** The primary enrichment method uses the **GitHub GraphQL API**. Instead of requesting data for every event individually, the **Enricher** accumulates up to 20 items (users or repositories) and fetches them in a single batched request. This significantly reduces the number of required HTTP calls and maximizes the utility of each API token.

- **REST Fallback Mechanism:** While GraphQL is the preferred method for efficiency, the system includes a robust **REST API fallback**. If a GraphQL batch query returns null for specific entities or encounters a partial failure, the system automatically falls back to individual REST calls to ensure that no data point is missed.
- **Geocoding (Geocoder & Photon):** Geographic data is processed by a dedicated geocoder service. It extracts location strings from enriched user profiles and queries the local **Photon** instance. This allows the system to resolve coordinates and country codes in the background without blocking the main event-processing pipeline.

### 3.4 Data Storage (TimescaleDB)

The project uses **TimescaleDB** (a PostgreSQL extension) as its primary storage solution:

- **Hypertables:** Event data is automatically partitioned into time-based chunks, which significantly improves query performance for large datasets.
- **Continuous Aggregates:** The database automatically precalculates rollups (e.g., 5-minute or 1-hour statistics) in the background, allowing complex dashboard queries to load in milliseconds. **[TODO: Add db scheme graph]**

### 3.5 Visualization

The primary platform for data analysis and system oversight is Grafana. It serves as the central visualization hub, providing a comprehensive suite of dashboards that directly query the TimescaleDB continuous aggregates and status logs. The dashboards are categorized into two main areas:

- **GitHub Data Insights (Hidden Gem Series):** These dashboards provide deep-dive analytics into the enriched GitHub data:
  - *Hidden Gem - Overview:* A high-level summary of all captured events and geographic distributions.
  - *Hidden Gem - Organizations:* Metrics focused on organizational activity and trends.
  - *Hidden Gem - Repositories:* Analysis of repository growth, popularity, and activity.
  - *Hidden Gem - Users:* Insights into user contributions and profiles.
- **System Monitoring and Status:** Dedicated dashboards track the operational health and performance of the pipeline:
  - *System Status Overview:* Monitoring of database performance and overall system health.
  - *Geocoder Request Logs:* Real-time tracking of the geocoding service's efficiency and request volume.
  - *General Request Logs:* Logs and metrics related to internal data flow and API interactions.

### 3.6 Geocoding Engine (Photon)

To implement high-performance geographic enrichment of GitHub events without relying on external rate-limited services, **Photon** was integrated into the infrastructure.

- **Local Instance:** Photon is an open-source geocoding service based on OpenStreetMap data. By operating it as a local Docker container, latency is minimized and the strict usage constraints of public APIs (such as Nominatim) are bypassed.
- **Data Basis and Regions:** The service is initialized with specific datasets—for instance, covering regions like Africa—which are persisted in the photon\_data volume to ensure stability and speed.
- **API Interface:** Photon provides a REST API that is queried by the geocoder service. It processes unstructured location strings from GitHub profiles and returns structured JSON data, including latitude, longitude, and ISO country codes.
- **Fault Tolerance:** The local instance’s high performance allows for complex fuzzy search queries. This ensures that even if a location cannot be resolved immediately, the overall data flow of the pipeline is not delayed.

Table 1: Expanded System Component Overview (Docker Services)

| Service     | Function                                                                             |
|-------------|--------------------------------------------------------------------------------------|
| producer    | Polls the GitHub API and writes raw event data to Kafka.                             |
| kafka       | Central message broker used for service decoupling and buffering.                    |
| consumer    | Enriches events with user and repository metadata via the GitHub API.                |
| geocoder    | Logic layer that coordinates location queries between the database and the engine.   |
| photon      | Geocoding engine that resolves location strings into geographic coordinates locally. |
| db-writer   | Consumes status and rate limit topics to persist operational metrics.                |
| timescaledb | Time-series optimized SQL database used for data storage and aggregation.            |
| grafana     | Central visualization platform for data insights and system monitoring.              |

4 IMPLEMENTATION

4.1 Data Ingestion

The producer (producer/producer.py) fetches up to three pages of the GET /events endpoint per poll cycle, yielding approximately 90 events every 10 seconds. An in-process set of recently seen event IDs prevents duplicate Kafka messages when polling overlaps the GitHub API cache window. Providing a GitHub personal access token raises the unauthenticated rate limit from 60 to 5 000 requests per hour [2].

4.2 Event Enrichment

The consumer (consumer/consumer.py) performs two enrichment passes:

- (1) **User-profile enrichment.** The GitHub REST API supplies each actor’s company name, self-reported location, public repository count, and follower count. Results are cached in a dictionary for the process lifetime to avoid redundant API calls.
- (2) **Geocoding.** Free-text location strings are resolved to WGS-84 coordinates via Nominatim [4]. A rate limiter enforces the required 1 req/s ceiling; a one-hour LRU cache avoids re-geocoding identical strings.

Enriched records are written to five normalised tables: users, repos, organizations, organization\_members, and events.

4.3 Storage Layer

The events table is a TimescaleDB hypertable partitioned into one-day chunks. Chunk-level columnar compression is applied after seven days (order by time DESC, segment by event\_type), achieving compression ratios of 10–20× over the raw row-store [6]. A 90-day data retention policy bounds disk usage.

Three continuous aggregates pre-compute rollups at ingest time rather than query time (Table 2), keeping dashboard query latency below a few milliseconds even at scale.

Table 2: TimescaleDB continuous aggregates

| View           | Bucket | Content                          |
|----------------|--------|----------------------------------|
| event_stats_5m | 5 min  | Event counts by event_type       |
| country_ids_5m | 5 min  | Event counts by ISO country code |
| actor_stats_1h | 1 h    | Per-actor event totals           |

4.4 API Layer

The FastAPI service exposes the endpoints listed in Table 3. The /stream/events endpoint implements the Server-Sent Events protocol [3]: it polls the v\_recent\_events database view every 3 seconds and writes JSON-encoded event objects to the response stream. Keep-alive comments (: ping) are sent between data frames to prevent browser and proxy timeouts.

Table 3: FastAPI endpoint reference

| Endpoint                   | Description                                |
|----------------------------|--------------------------------------------|
| GET /health                | Liveness check                             |
| GET /api/kpis              | KPI metrics with 30-day deltas             |
| GET /api/commits-over-time | Daily commit buckets (configurable window) |
| GET /api/top-repos         | Most active repositories                   |
| GET /api/recent-events     | Latest ingested events                     |
| GET /stream/events         | SSE live stream (3 s poll)                 |

4.5 Visualisation

The Next.js [7] dashboard (port 3000) uses React 19 server components to fetch KPI and chart data at request time via the API\_URL environment variable (internal Docker DNS). Client components subscribe to the SSE stream using a typed useSSE React hook that reconnects automatically after a 3-second back-off. The UI is built with Tremor v3 (Recharts under the hood) and Tailwind CSS 4.

5 EVALUATION

**TODO:** Fill in this section after a representative system run (24–48 hours recommended). Suggested structure below.

### 5.1 Experimental Setup

*5.1.1 Hardware and Operating System.* The entire pipeline was hosted on a dedicated virtualized server. The system leverages high-performance AMD EPYC processors and sufficient memory to handle the concurrent execution of multiple Docker containers, including the Kafka broker and the TimescaleDB instance. The detailed system specifications are summarized in Table 4.

Table 4: System Specifications

| Component        | Specification                       |
|------------------|-------------------------------------|
| Hostname         | github-insights                     |
| Operating System | Ubuntu 24.04.4 LTS (Noble)          |
| Kernel           | Linux 6.17.0-20-generic (x86_64)    |
| CPU              | AMD EPYC-Milan (16 vCPUs, 16 Cores) |
| Hypervisor       | KVM (Full Virtualization)           |
| Memory           | 31 GiB Total                        |
| Storage          | 348 GB (NVMe-based Virtual Disk)    |

*5.1.2 Software Environment and Containerization.* The architecture is fully containerized using **Docker** and managed via **Docker Compose**. This approach ensures service isolation and simplified dependency management. The primary software components and their respective versions are:

- **Message Broker:** Apache Kafka v3.9.0, running in KRaft mode for metadata management without Zookeeper.
- **Database:** TimescaleDB (based on PostgreSQL 16), utilizing hypertables and continuous aggregates for time-series optimization.
- **Geocoding Engine:** Photon (OpenStreetMap-based) running as a local REST service.
- **Runtime Environment:** Python 3.11+ for the Producer, Consumer, and Geocoder microservices.

*5.1.3 Pipeline Configuration.* For the experimental runs, the system was configured in **multi-consumer mode** to test horizontal scalability.

- **Parallelism:** Three independent consumer instances were deployed, each pinned to a specific Kafka partition to prevent processing overlaps.
- **Resource Allocation:** Given the 16 vCPUs available, the load was distributed across the Kafka broker, the TimescaleDB background workers, and the enrichment consumers to maintain a load average below 1.0 during peak ingestion.
- **Network:** A bridge-driver Docker network with a custom MTU of 1442 was used to optimize internal container communication.

### 5.2 Throughput and Latency

Table 5 summarises the key performance metrics measured during the evaluation period.

### 5.3 Data Coverage and Enrichment Quality

Table 6 shows the breadth of the data collected.

Table 5: Pipeline throughput and latency metrics

| Metric                      | Value            | Notes                     |
|-----------------------------|------------------|---------------------------|
| Events ingested (total)     | [TODO: N]        | Over a [TODO: T]-hour run |
| Average ingestion rate      | [TODO: X] ev/min |                           |
| Peak ingestion rate         | [TODO: X] ev/min |                           |
| Median end-to-end latency   | [TODO: X] s      | Poll → dash-board update  |
| Kafka consumer lag (steady) | [TODO: X] msgs   | Measured via Kafka-UI     |

Table 6: Data coverage over evaluation period

| Dimension                     | Value                                     |
|-------------------------------|-------------------------------------------|
| Unique repositories           | [TODO: N]                                 |
| Unique contributors           | [TODO: N]                                 |
| Countries represented         | [TODO: N]                                 |
| Geocoding success rate        | [TODO: X]%                                |
| Event types observed          | [TODO: list, e.g. PushEvent, PullRequest] |
| Most active programming lang. | [TODO: language]                          |

### 5.4 Storage Efficiency

[TODO: Report uncompressed vs. compressed chunk size for the events hypertable. Query: SELECT \* FROM chunk\_compression\_stats('events');

## 6 TECHNOLOGY DECISION RATIONALE

*Apache Kafka.* Kafka [1] decouples the rate-limited GitHub poller from the slower geo-enrichment step. Its consumer group model allows the enrichment workload to be parallelised by running additional consumer containers without modifying the producer. The durable log acts as a replay buffer: if the consumer crashes mid-processing, it resumes from its last committed offset without data loss.

*TimescaleDB over plain PostgreSQL..* Plain PostgreSQL lacks automatic time-based chunk pruning and columnar compression; queries over large event tables become progressively slower as data accumulates. TimescaleDB [6] adds hypertables (transparent partitioning by time), 10–20× compression, and continuous aggregates that materialise rollups at ingest time. It retains full SQL compatibility, which is essential for the JOIN-heavy enrichment queries and JSONB payload fallback.

*FastAPI over Flask/Django.* FastAPI’s [5] async-first design and native StreamingResponse make SSE implementation straightforward. The automatic OpenAPI documentation at /docs was valuable during development and testing.



*Next.js 16 (App Router).* Server components eliminate the waterfall of client-side data fetches on initial page load. The App Router's layout system provides a persistent sidebar without duplicating markup across pages. Tremor's pre-built chart components (AreaChart, BarChart, Table) accelerated dashboard development significantly.

## 7 CHALLENGES AND SOLUTIONS

*GitHub API rate limiting.* Without authentication, the REST API caps unauthenticated requests at 60 per hour — insufficient for continuous polling. We mitigated this with an optional personal access token (raising the cap to 5 000/h) and client-side deduplication that avoids re-publishing events already seen within the polling window.

*Geocoding throughput.* Nominatim enforces a strict 1 req/s policy for fair use. We introduced an in-process LRU cache keyed on the normalised location string, which eliminated redundant lookups for common values such as “San Francisco, CA” or “Berlin, Germany”.

*Service startup ordering.* TimescaleDB and Kafka require 10–30 seconds after container start before accepting connections. Docker Compose health checks gate dependent services. Additionally, the FastAPI service implements an internal retry loop that polls the database connection every two seconds and logs progress until the pool is established.

*React 19 / Tremor peer dependency conflict.* Tremor v3 declares a React 18 peer dependency while the project uses React 19. Installing with `-legacy-peer-deps` bypasses the version check; no runtime incompatibilities were observed during development.

**[TODO: Additional challenges.] [TODO: Add any further challenges encountered during the project, e.g., Kafka KRaft configuration, TimescaleDB compression policy tuning, SSE browser compatibility, etc.]**

## 8 CONCLUSION AND FUTURE WORK

**[TODO: Write 1–2 paragraphs. Summarise what was achieved relative to the three project objectives: (O1) defining momentum metrics, (O2) building a live dashboard, and (O3) granular filtering. Mention the most interesting finding from the data.]**

The GitHub Innovation Portal demonstrates that a small team can build a production-grade, end-to-end streaming analytics platform using entirely open-source components within a single academic semester. The combination of Kafka for durable buffering, TimescaleDB for time-series storage, and a modern React frontend proved well-suited to the high-velocity, enrichment-heavy nature of the GitHub event stream.

Several directions remain for future work:

- **Stream analytics.** Attaching Apache Flink or Spark Structured Streaming to the `github.events.raw` topic would enable windowed anomaly detection (e.g., sudden starring spikes) without modifying the existing pipeline.
- **Granular filtering (O3).** Extending the FastAPI query layer with language, licence, and topic filter parameters and wiring

these into a dashboard filter panel would complete the third project objective.

- **Long-term archival.** A Kafka Connect S3 sink writing Parquet files to object storage would enable cost-efficient historical analysis beyond the 90-day TimescaleDB retention window.
- **Alerting.** Grafana alerting rules on `event_stats_5m` could notify users when a repository's event velocity exceeds a configurable threshold — a first step towards automated “rising star” detection.

## ACKNOWLEDGMENTS

The authors thank the PM4 module supervisors at ZHAW School of Engineering for their guidance throughout the project. **[TODO: Add names of supervisors or any other individuals to acknowledge.]**

## REFERENCES

- [1] Apache Software Foundation. 2025. Apache Kafka Documentation. <https://kafka.apache.org/documentation/>. Accessed: March 2025.
- [2] GitHub, Inc. 2025. GitHub REST API Documentation. <https://docs.github.com/en/rest>. Accessed: March 2025.
- [3] Ian Hickson. 2015. Server-Sent Events — W3C Recommendation. <https://www.w3.org/TR/eventsource/>. W3C Recommendation, 3 February 2015.
- [4] OpenStreetMap contributors. 2025. Nominatim Search API Documentation. <https://nominatim.org/release-docs/latest/api/Search/>. Accessed: March 2025.
- [5] Sebastián Ramírez. 2025. FastAPI Documentation. <https://fastapi.tiangolo.com/>. Accessed: March 2025.
- [6] Timescale, Inc. 2025. TimescaleDB Documentation. <https://docs.timescale.com/>. Accessed: March 2025.
- [7] Vercel, Inc. 2025. Next.js Documentation. <https://nextjs.org/docs>. Accessed: March 2025.

A PROJECT TIMELINE

Table 7: Actual project timeline and task allocation

| Week  | Phase        | Deliverable                                                 | Owner           |
|-------|--------------|-------------------------------------------------------------|-----------------|
| KW 03 | Setup        | Kafka pipeline, TimescaleDB schema, Docker Compose skeleton | [TODO: name(s)] |
| KW 04 | Exploration  | GitHub event stream collection, EDA, schema refinement      | [TODO: name(s)] |
| KW 06 | Pipeline     | Producer, consumer, geo-enrichment, FastAPI endpoints       | [TODO: name(s)] |
| KW 10 | Dashboard    | Next.js dashboard, Grafana provisioning, SSE live feed      | [TODO: name(s)] |
| KW 13 | Finalization | Testing, documentation, final report, presentation          | All             |

B DATABASE SCHEMA

Table 8: Relational schema (simplified)

| Table                | PK                         | Key columns                                                                                                                    |
|----------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| events               | (time, event_id)           | event_type, actor_username, repo_id, detail, payload (JSONB). Hy-per-table: 1-day chunks; 7-day compression; 90-day retention. |
| users                | username                   | company, location, country, country_code, lat, lng, public_repos, followers.                                                   |
| repos                | repo_id                    | full_name, language, topics[], stargazers_count, forks_count.                                                                  |
| organizations        | login                      | name, location, is_verified.                                                                                                   |
| organization_members | (org_login, user_username) | role.                                                                                                                          |

C DEPLOYMENT QUICK REFERENCE

```
# 1. Clone and configure
git clone <repo-url> && cd pm4-github-insights
cp .env.example .env # add GITHUB_TOKEN

# 2. Start all services
docker compose up --build

# 3. Access UIs
```

```
# http://localhost:3000 Next.js dashboard
# http://localhost:8000/docs FastAPI Swagger UI
# http://localhost:3001 Grafana (admin / admin)
# http://localhost:8080 Kafka-UI
```

```
# 4. Open a DB session
```

```
docker exec -it timescaledb \
psql -U github -d github_events
```

See docs/RUNBOOK.md and docs/ENV.md for the full operational reference and environment variable documentation.

Received January 2025; revised [TODO: Fill in revision date]; accepted March 2025